// Day 11 of 30

STORY + LESSON

How a Single SQL Query Crashed Our Production Server

→ Swipe through all 6 slides

Shubham Bhati
linkedin.com/in/bhatishubham

Slide 01

The Innocent Query

It looked harmless. It wasn't.

```
SELECT * FROM transactions
WHERE created_at > '2025-01-01'
ORDER BY amount DESC;

-- Looked fine in dev (100 rows)
-- Production had 8 MILLION rows
```

💡 Always test queries with production-sized data

Shubham Bhati
linkedin.com/in/bhatishubham

Slide 02

What Happened Next

3 AM. PagerDuty goes off.

```
CPU: 98%
Memory: 95%
Response time: 47 seconds
Active connections: maxed out

-- One query locked the entire table
-- Every other query queued behind it
```

💡 One bad query can take down everything

Why It Crashed

No index. Full table scan.

```
EXPLAIN ANALYZE showed:
-- Seq Scan on transactions
-- Rows examined: 8,247,312
-- Time: 43,291ms

No index on created_at column
No LIMIT clause
SELECT * instead of specific columns
```

💡 EXPLAIN ANALYZE is your best friend

The 5-Minute Fix

Three changes. Problem solved.

```
1. Added index:
CREATE INDEX idx_txn_date
  ON transactions(created_at);

2. Added LIMIT:
LIMIT 1000

3. Selected specific columns:
SELECT id, amount, status
```

💡 Index + LIMIT + specific columns = safe

Shubham Bhati
linkedin.com/in/bhatishubham

Before vs After

Numbers don't lie

```
BEFORE:
  Query time: 43,291ms
  Rows scanned: 8,247,312
  CPU: 98%

AFTER:
  Query time: 12ms
  Rows scanned: 1,000
  CPU: 3%
```

💡 3,600x faster with 3 simple changes

Lessons Learned

Never again.

```
1. Never use SELECT * in production
2. Always add indexes on WHERE columns
3. Always use LIMIT on large tables
4. Run EXPLAIN before deploying queries
5. Test with production-sized datasets
6. Set up query timeout limits
```

💡 Prevention > firefighting at 3 AM

```
done = true
```

Follow for more!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham